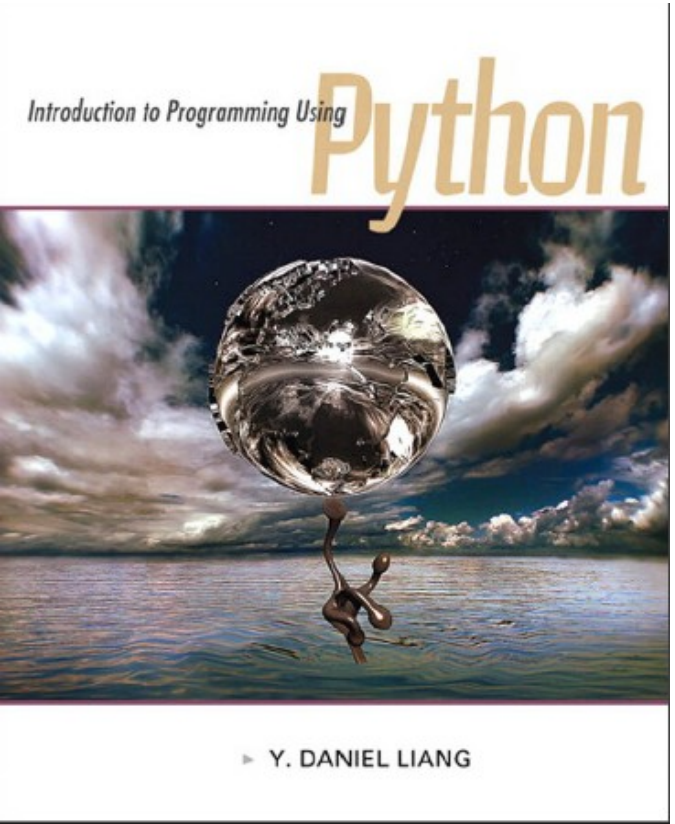


Chapter 04 Selections



Prepared By:

Eng. Eyad J. R. Elfarra

Objectives

- To write Boolean expressions by using comparison operators (§4.2).
- To generate random numbers by using the **random.randint(a, b)** or **random.random()** functions (§4.3).
- To program with Boolean expressions (**AdditionQuiz**) (§4.3).
- To implement selection control by using one-way if statements (§4.4)
- To program with one-way if statements (**GuessBirthday**) (§4.5).
- To implement selection control by using two-way **if .. else** statements (§4.6).
- To implement selection control with nested **if … elif … else** statements (§4.7).
- To avoid common errors in **if** statements (§4.8).
- To program with selection statements (§4.9–4.10).
- To combine conditions by using logical operators (**and**, **or**, and **not**) (§4.11).
- To use selection statements with combined conditions (**LeapYear**, **Lottery**) (§§4.12–4.13).
- To write expressions that use the conditional expressions (§4.14).
- To understand the rules governing operator precedence and associativity (§4.15).

4.1 Introduction

A program can decide which statements to execute based on a condition.

Motivation

- If you assigned a negative value for the radius of the circle then the **circumference** of the circle will be negative. Is this true?
- If the radius is negative, you don't want the program to compute the circumference.
- **How can you deal with this situation?**
- Python provides **selection statements** that let you choose actions with two or more alternative courses.

```
if radius < 0:
  print("Incorrect input")
else:
   circumference = 2 * radius * math.pi
print(“Circumference is", circumference)
```

Motivation

- Selection statements use conditions.
- Conditions are Boolean expressions.
- This chapter introduces:
  - Boolean types.
  - Boolean values.
  - Comparison operators, and expressions.

4.2 Boolean Types, Values, and Expressions

A Boolean expression is an expression that evaluates to a Boolean value True *or* False*.*

Boolean Data Types

- Often in a program you need to **compare** two values, such as whether **i** is *greater* than **j**.
- There are **six** comparison operators (also known as **relational operators**) that can be used to compare two values.
- The **result** of the comparison is a **Boolean value**: *True* or *False*.

```
b = (1 > 2)
```

Comparison Operators

- **Boolean expressions** ask a question and produce a **Yes** or **No** result which we use to control program flow
- **Boolean expressions** using ***comparison operators*** evaluate to True / False or Yes / No

  Comparison operators look at variables but **do not change** the variables

Python	Meaning
<	Less than
<=	Less than or Equal to
==	Equal to
>=	Greater than or Equal to
>	Greater than
!=	Not equal

Remember: “=” is used for assignment.

Comparison Operators

TABLE 4.1 Comparison Operators

Python Operator	*Mathematics Symbol*	*Name*	*Example* (radius is 5)	*Result*
`<`	<	less than	`radius < 0`	`False`
`<=`	≤	less than or equal to	`radius <= 0`	`False`
`>`	>	greater than	`radius > 0`	`True`
`>=`	≥	greater than or equal to	`radius >= 0`	`True`
`==`	=	equal to	`radius == 0`	`False`
`!=`	≠	not equal to	`radius != 0`	`True`

Comparison Operators

```
x = 5
if x == 5 :
    print('Equals 5')
if x > 4 :
   print('Greater than 4')
if  x >= 5 :
    print('Greater than or Equals 5')
if x < 6 : print('Less than 6')
if x <= 5 :
    print('Less than or Equals 5')
if x != 6 :
    print('Not equal 6')
```

Equals 5

Greater than 4

Greater than or Equals 5

Less than 6

Less than or Equals 5

Not equal 6

Boolean

- A **variable** that holds a **Boolean** value is known as a ***Boolean variable***.
- A Boolean variable can hold one of the two values:
  - True or False.
  - They are reserved words and cannot be used as identifiers in a program.

```
>>> type(True)
<class 'bool'>
```

- **Internally**, Python uses **1** to represent True and **0** for False.
  - You can use the int function to convert a Boolean value to an integer.

```
>>> print(int(True))
1
>>> print(int(False))
0
>>>
```

bool function

- bool(obj) function **converts** the object to a Boolean value.
- The function returns
  - **False:** if the value is
    - 0, [],(),{}, False, None(empty)
  - **True**:
    - otherwise

```
>>>bool(1)
True
>>>bool(0)
False
>>>bool(25)
True
>>>bool(-10)
True
>>>bool()
False
>>>bool("vv")
True
```

4.3 Generating Random Numbers

randint function

- The randint(start,end) method:
  - Is a built-in function from the random module
  - Generate a random integer between **start** and **end**.
  - Start & end are **included** in the range

- Syntax

```
random.randint(start, stop)
```

- Parameter Values
  - **start**: Required. An integer specifying at which position to start.
  - **end**: Required. An integer specifying at which position to end.

```
>>>import random
>>>random.randint(0,5)
2
>>>random.randint(0,5)
5
>>>random.randint(0,5)
1
```

Problem: A Simple Math Learning Tool

- This example creates a program to let a first grader practice additions. The program **randomly** generates **two single-digit integers** number1 and number2 and **displays** a question such as “What is 7 + 9?” to the student. After the student **types** the answer, the program **displays** a message to indicate whether the answer is true or false.

Problem: A Simple Math Learning Tool

- The program may be set up to work as follows:
- **Step 1**: Generate two single-digit integers for number1 (e.g., 4) and number2 (e.g., 5)

```
# Generate random numbers
number1 = random.randint(0, 9)
number2 = random.randint(0, 9)
```

- **Step 2**: Prompt the student to answer "What is 4 + 5?"

```
# Prompt the user to enter an answer
answer = eval(input("What is " + str(number1) + " + "
    + str(number2) + "? "))
```

- **Step 3**: Check whether the student's answer is correct

```
# Display result
print(number1, "+", number2, "=", answer,
    "is", number1 + number2 == answer)
```

Another random functions

- randrange(a, b):
  - generating a random integer between a and b – 1
  - which is equivalent to randint(a, b – 1)
  - random.randrange(0, 1)
    - will always be 0

- random() function:
  - generate a random float r such that 0 <= r <1.0

```
>>> import random
>>> random.random()
0.34343
>>> random.random()
0.20119
>>> random.randint(0, 1)
0
>>> random.randint(0, 1)
1
>>> random.randrange(0, 1)
0
>>>
```

4.4 if Statements

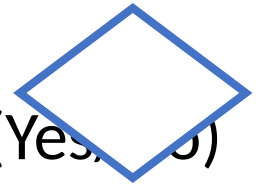
A one-way if *statement executes the statements if the condition is true.*

Python selection Statements

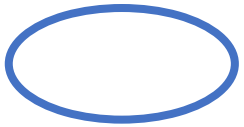
- Python has several types of selection statements:
  - One-way if statements,
  - Two-way if else statements
  - Nested if statements
  - Multi-way if-elif-else statements
  - Conditional expressions.
- An if statement executes statements if the **boolean-expression** evaluates to **True**.

Flowchart

- A flowchart is a diagram that describes an algorithm or process, showing the **steps** as boxes of various kinds, and their order by connecting these with arrows.
- Process **operations** are represented in these boxes, and arrows connecting them show flow of control.
- A **diamond box** is used to denote a **Boolean condition**. → Decision (Yes/No)



- **Oval** → Start / End



- **Rectangle** → Process (instructions)



- **Parallelogram** → Input / Output

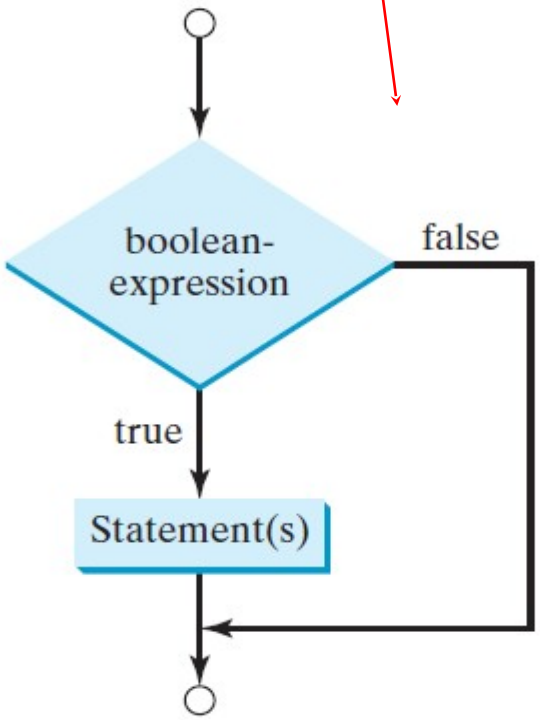


- **Arrow** → Flow of control

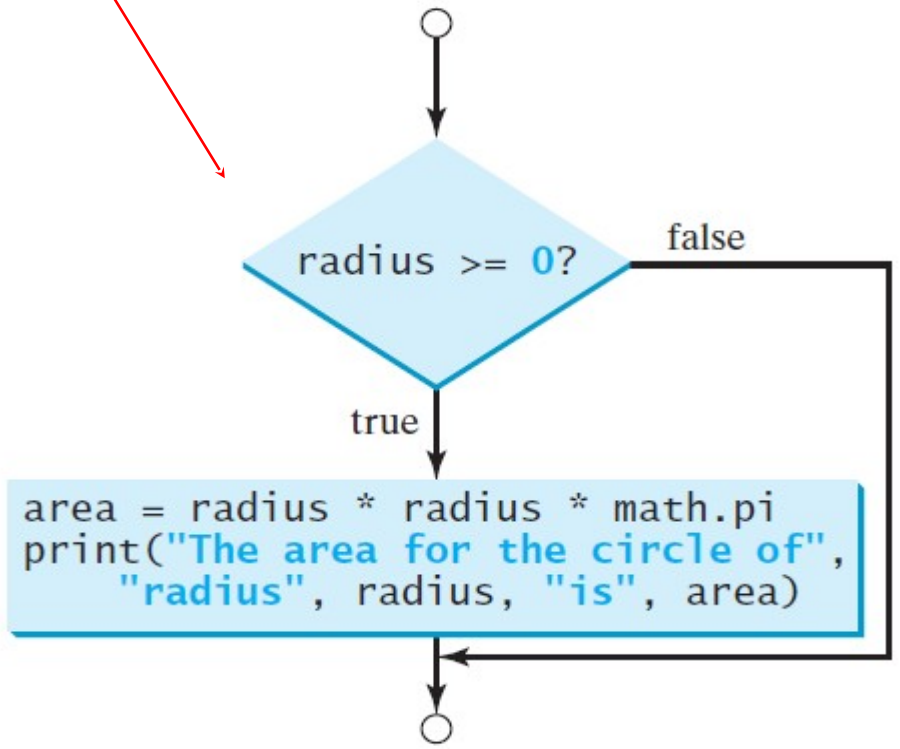


One-way `if` Statements

```
if boolean-expression:
    statement(s)
```



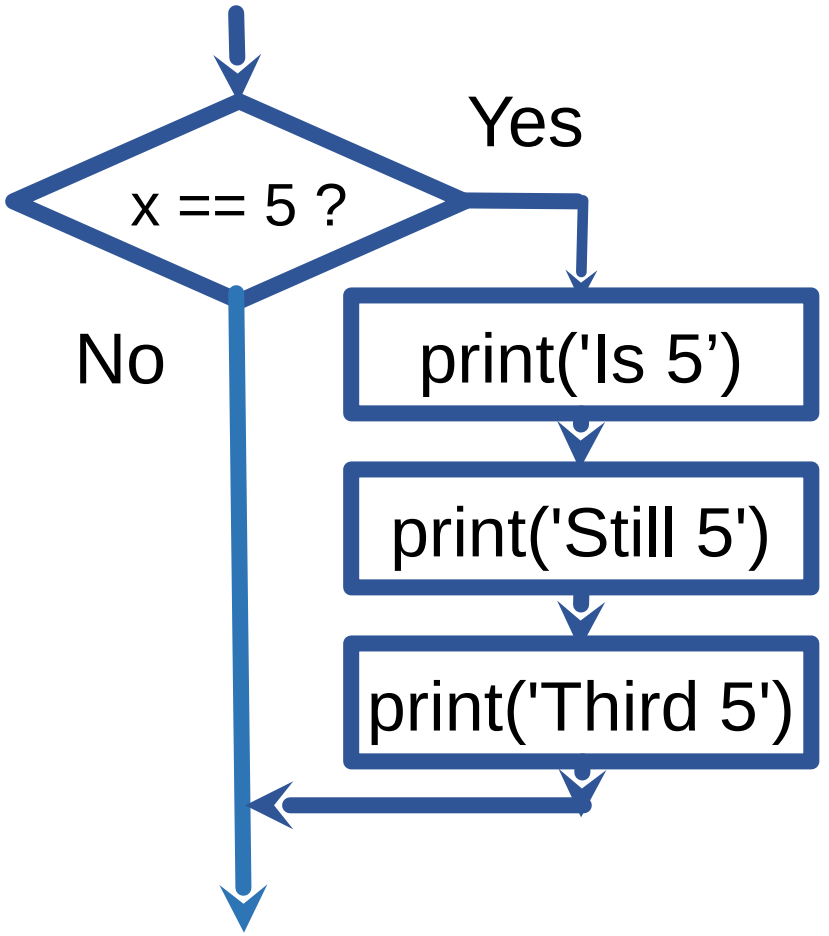

```
if radius >= 0:
    area = radius * radius * 3.14159
    print("The area for the circle of radius“,
          radius, "is“, area)
```



One-Way Decisions

```
print('Before 5')
if  x == 5 :
    print('Is 5')
    print('Is Still 5')
    print('Third 5')
print('Afterwards 5')
print('Before 6')
if x == 6 :
    print('Is 6')
    print('Is Still 6')
    print('Third 6')
print('Afterwards 6')
```

```
Before 5
Is 5
Is Still 5
Third 5
Afterwards 5
Before 6
Afterwards 6
```



Note

```
if i > 0:
print("i is positive")
```

(a) Wrong

```
if i > 0:
    print("i is positive")
```

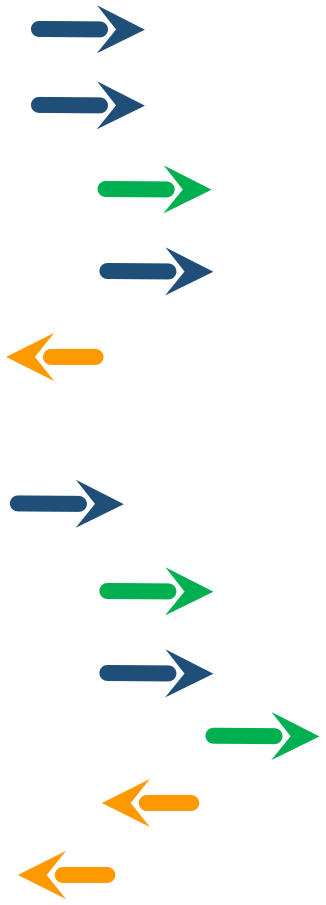
(b) Correct

Indentation

- **Increase indent** indent after an *if* statement or *for* statement (after :)
- **Maintain indent** to indicate the **scope** of the block (which lines are affected by the *if/for*)
- **Reduce indent** back to the level of the *if* statement or *for* statement to indicate the end of the block
- *Blank lines* are ignored - they do not affect **indentation**
- *Comments* on a line by themselves are ignored with regard to **indentation**

increase / maintain after if or for

decrease to indicate end of block



```
x = 5
if x > 2 :
    print('Bigger than 2')
    print('Still bigger')
print('Done with 2')

for i in range(5) :
    print(i)
    if i > 2 :
        print('Bigger than 2')
    print('Done with i', i)
print('All Done')
```

Think About begin/end Blocks

```
x = 5
if x > 2 :
    print('Bigger than 2')
    print('Still bigger')
print('Done with 2')

for i in range(5) :
    print(i)
    if i > 2 :
        print('Bigger than 2')
    print('Done with i', i)
print('All Done')
```

Simple if Demo

- Write a program that **prompts** the user to enter an integer. If the number is a multiple of 5, print HiFive. If the number is divisible by 2, print HiEven.
- Steps:
  - **Step 1**: ask user to enter the number

```
number = eval(input("Enter an integer: "))
```

  - **Step 2**: Check if the number is divisible by 5, then print **HiFive**

```
if number % 5 == 0:
    print("HiFive")
```

  - **Step 3**: Check if the num[illegible] then print **HiEven**

```
if number % 2 == 0:
    print("HiEven")
```

SimpleIfDemo.py

```
number = eval(input("Enter an integer: "))

if number % 5 == 0:
 print("HiFive")

if number % 2 == 0:
 print("HiEven")
```

```
Enter an integer: 4 ↵Enter
HiEven
```

```
Enter an integer: 30 ↵Enter
HiFive
HiEven
```

4.5 Case Study: Guessing Birthdays

Guessing birthdays is an interesting problem with a simple program solution.

line continuation symbol (\)

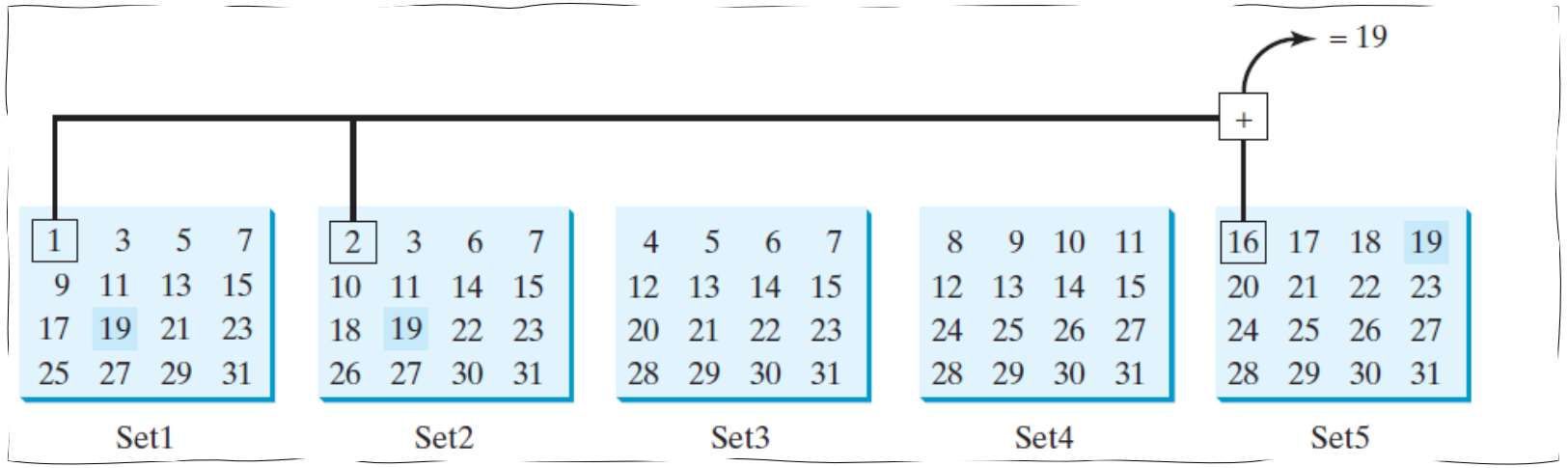
- In some cases, the Python interpreter cannot determine the end of the statement written in multiple lines.
- You can place the line continuation symbol (\) at the end of a line to tell the interpreter that the statement is continued on the next line.
- For example, the following statement

```
sum = 1 + 2 + 3 + 4 + \
      5 + 6
```

```
sum = 1 + 2 + 3 + 4 + 5 + 6
```

Problem: Guessing Birthday

- The program can guess your birth date.
- You can find out the date of the month when your friend was born by **asking five questions**.
- Each **question** asks **whether** the **day is in one of the five sets** of numbers.
- The birthday is the sum of the first numbers in the sets where the date appears.

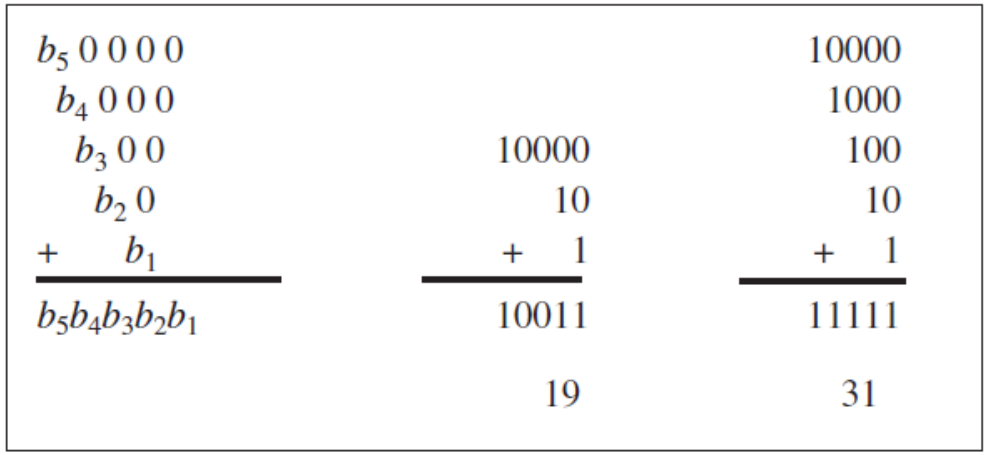


Mathematics Basis for the Game

- The mathematics behind the game is actually quite simple. The numbers **are not grouped together by accident**. The way they are placed in the five sets is deliberate.
- The starting numbers in the five sets are 1, 2, 4, 8, and 16, which correspond to 1, 10, 100, 1000, and 10000 in binary.
- A binary number for decimal integers between 1 and 31 has at most five digits, as shown in Figure 4.2a. Assume this number $b5b4b3b2b1$.is So, $b5b4b3b2b1 = b50000 + b4000 + b300 + b20 + b1$, as shown in Figure 4.2b.

Decimal	Binary
1	00001
2	00010
3	00011
...	
19	10011
...	
31	11111

(a)



(b)

GuessBirthday.py

```
Is your birthday in Set1?
 1  3  5  7
 9 11 13 15
17 19 21 23
25 27 29 31
Enter 0 for No and 1 for Yes: 1 ↵Enter

Is your birthday in Set2?
 2  3  6  7
10 11 14 15
18 19 22 23
26 27 30 31
Enter 0 for No and 1 for Yes: 1 ↵Enter

Is your birthday in Set3?
 4  5  6  7
12 13 14 15
20 21 22 23
28 29 30 31
Enter 0 for No and 1 for Yes: 0 ↵Enter

Is your birthday in Set4?
 8  9 10 11
12 13 14 15
24 25 26 27
28 29 30 31
Enter 0 for No and 1 for Yes: 0 ↵Enter
```

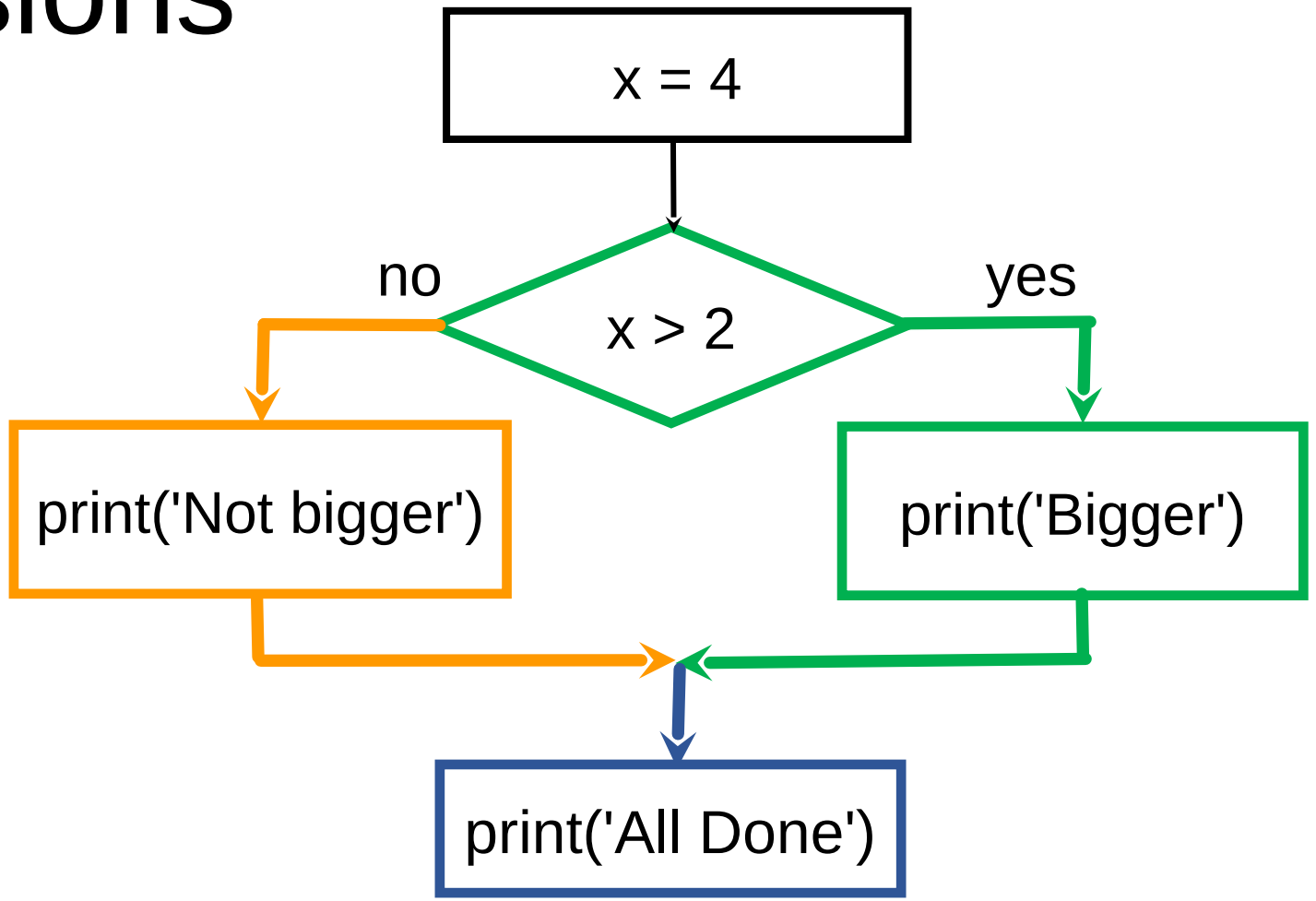
```
Is your birthday in Set5?
16 17 18 19
20 21 22 23
24 25 26 27
28 29 30 31
Enter 0 for No and 1 for Yes: 1 ↵Enter
Your birthday is 19!
```

4.6 Two-Way if-else Statements

Two-way Decisions

Sometimes we want to do one thing if a logical expression is true and something else if the expression is false

It is like a fork in the road - we must choose ***one or the other*** path but not both

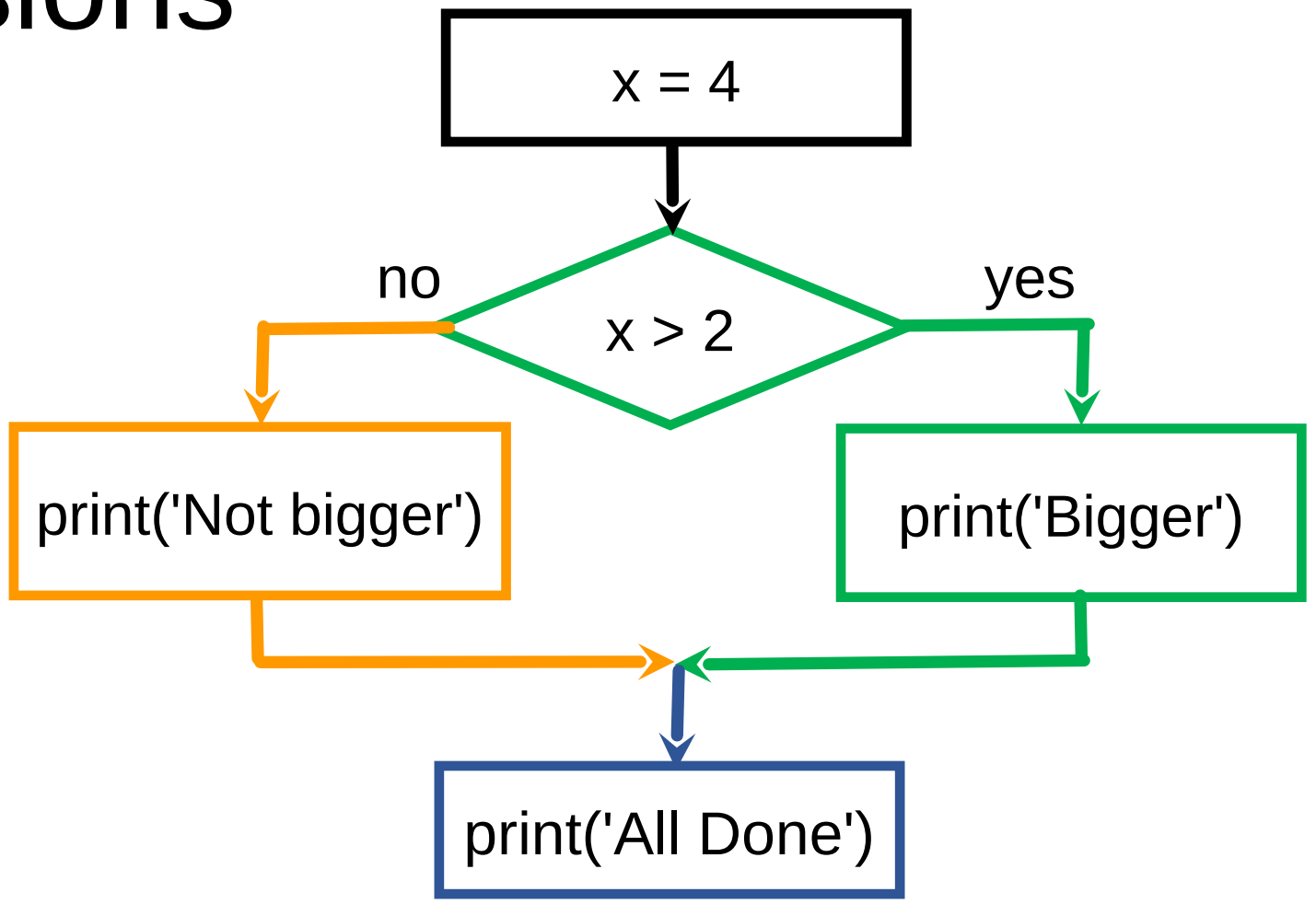


Two-way Decisions with else:

```
x = 4

if x > 2 :
    print('Bigger')
else :
    print('Smaller')

print('All done')
```

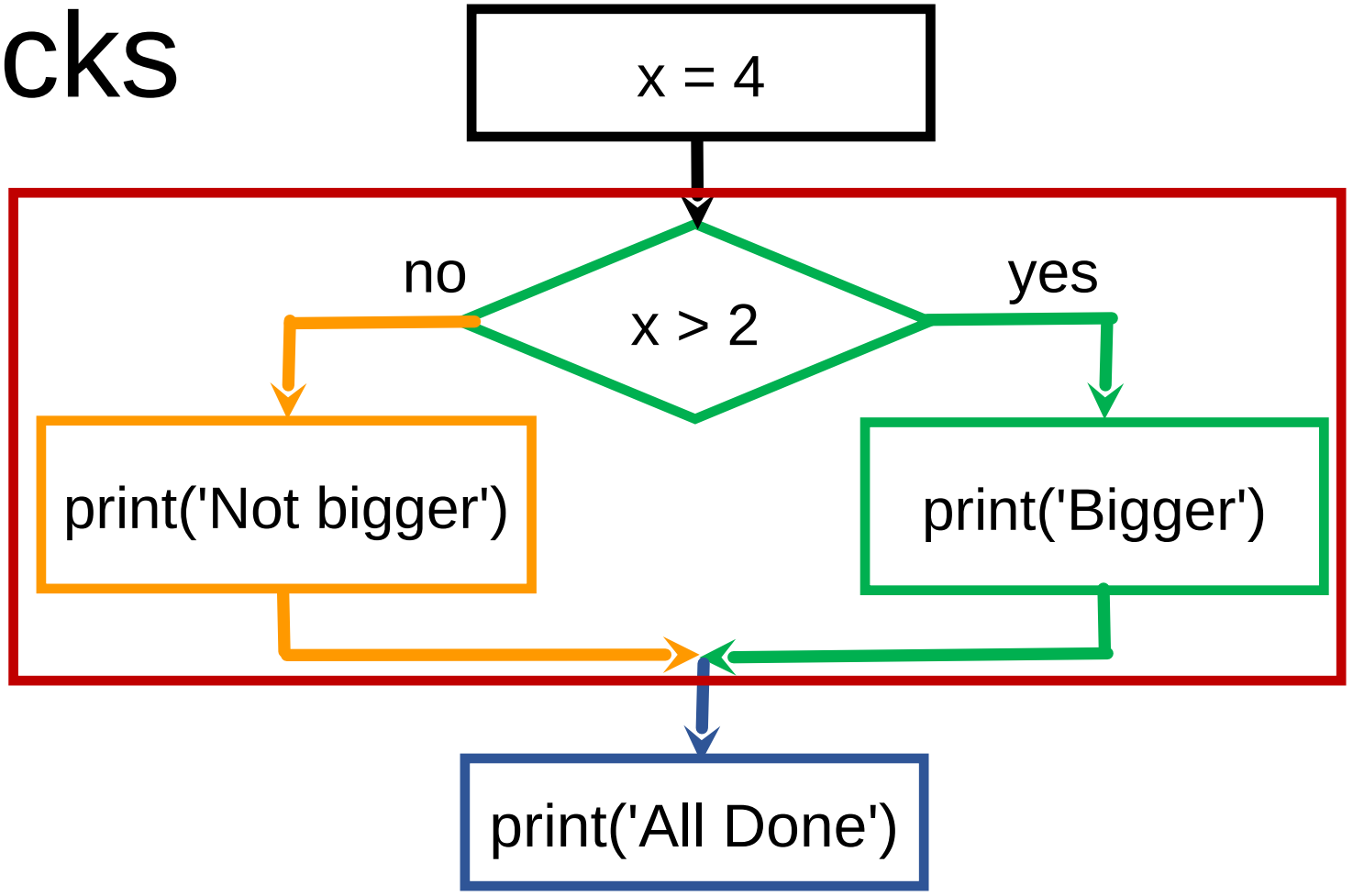


Visualize Blocks

```
x = 4

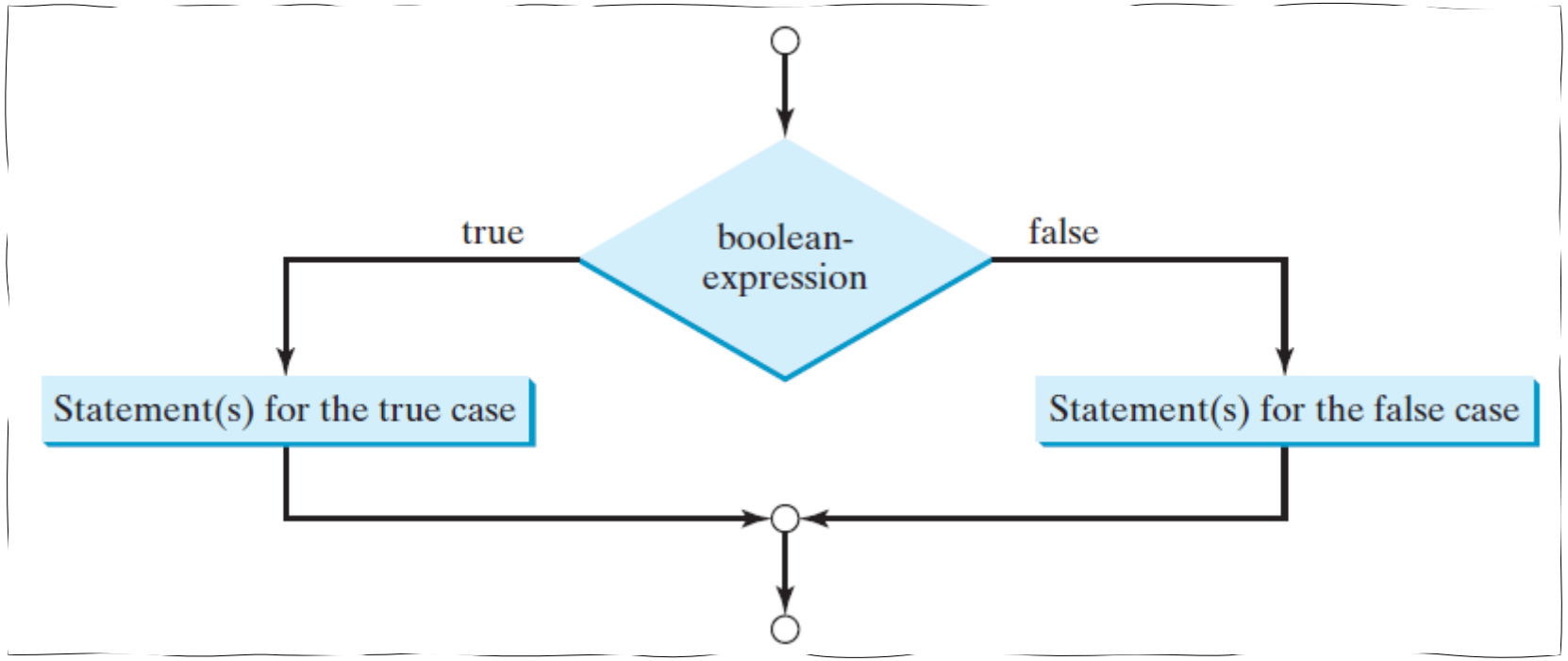
if x > 2 :
    print('Bigger')
else :
    print('Smaller')

print('All done')
```



The Two-way `if` Statement

```
if boolean-expression:
    statement(s)-for-the-true-case
else:
    statement(s)-for-the-false-case
```



If else Examples(EvenOdd.py)

```
number = eval(input("Enter an integer: "))

if number % 2 == 0:
    print(number, "is even.")
else:
    print(number, "is odd.")
```

First Run

```
Enter an integer: 5
5 is odd.
```

Second Run

```
Enter an integer: 4
4 is even.
```

If else Examples(ComputeCircleArea.py)

```
import math
radius = eval(input("Enter the radius: "))
if radius >= 0:
      area = radius * radius * math.pi
      print("The area for the circle of radius", radius, "is", area)
else:
      print("Negative input")
```

First Run

```
Enter the radius: 10
The area for the circle of radius 10 is 314.1592653589793
```

Second Run

```
Enter the radius: -10
Negative input
```

Problem: An Improved Math Learning Tool

- This example creates a program to teach a first grade child how to learn **subtractions**.
- The program **randomly** generates **two single-digit integers** number1 and number2 with number1 > number2
- and displays a question such as “What is 9 – 2?” to the student.
- After the student **types** the answer in the input dialog box, the **program displays** a message dialog box to indicate whether the **answer is correct**.

SubtractionQuiz.py

- The program may work as follows:
  - **Step 1:** Generate two single-digit integers for number1 and number2.

```
# 1. Generate two random single-digit integers
number1 = random.randint(0, 9)
number2 = random.randint(0, 9)
```

  - **Step 2**: If number1 < number2, swap number1 with number2.

```
# 2. If number1 < number2, swap number1 with number2
if number1 < number2:
    number1, number2 = number2, number1  # Simultaneous assignment
```

SubtractionQuiz.py

- The program may work as follows:
  - **Step 3**: Prompt the student to answer, “What is number1 – number2?”

```
# 3. Prompt the student to answer "What is number1 - number2?"
answer = eval(input("What is "+ str(number1) + " - " +
    str(number2) + "? "))
```

  - **Step 4**: Check the student’s answer and display whether the answer is correct.

```
# 4. Check the answer and display the result
if number1 - number2 == answer:
    print("You are correct!")
else:
    print("Your answer is wrong.\n", number1, '-',
        number2, "is", number1 - number2, '.')
```

4.7 Nested if and Multi-Way if-elif-else Statements

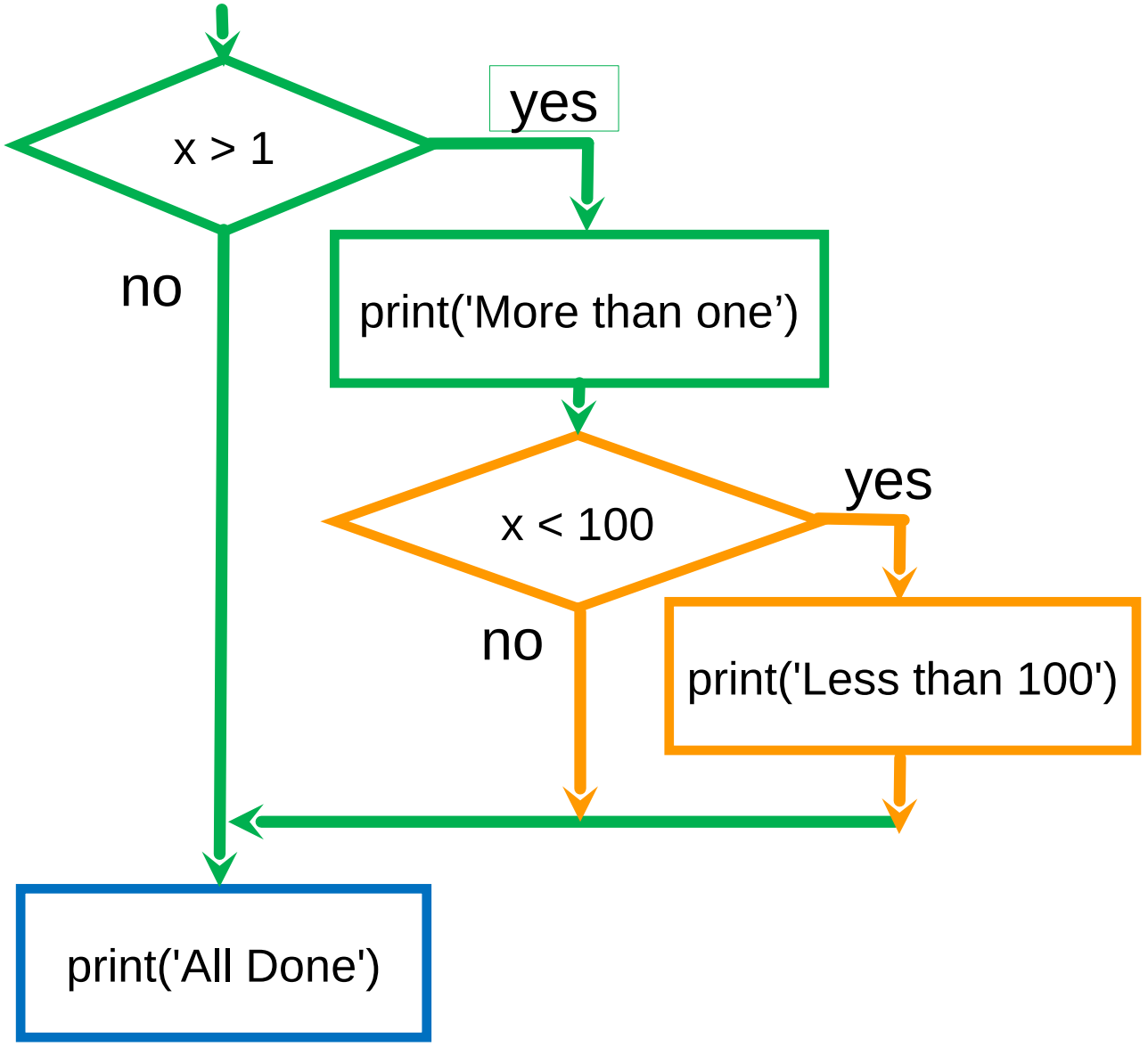
One if *statement can be placed inside another* if *statement to form a nested* if *statement.*

Nested if

- The statement in an **if** or **if-else** statement can be any legal Python statement, including:
  - Another if or if-else statement.
  - The **inner if** statement is said to be **nested** inside the outer if statement.
  - The inner if statement **can contain** another if statement;
- In fact, there is **no limit** to the depth of the nesting.

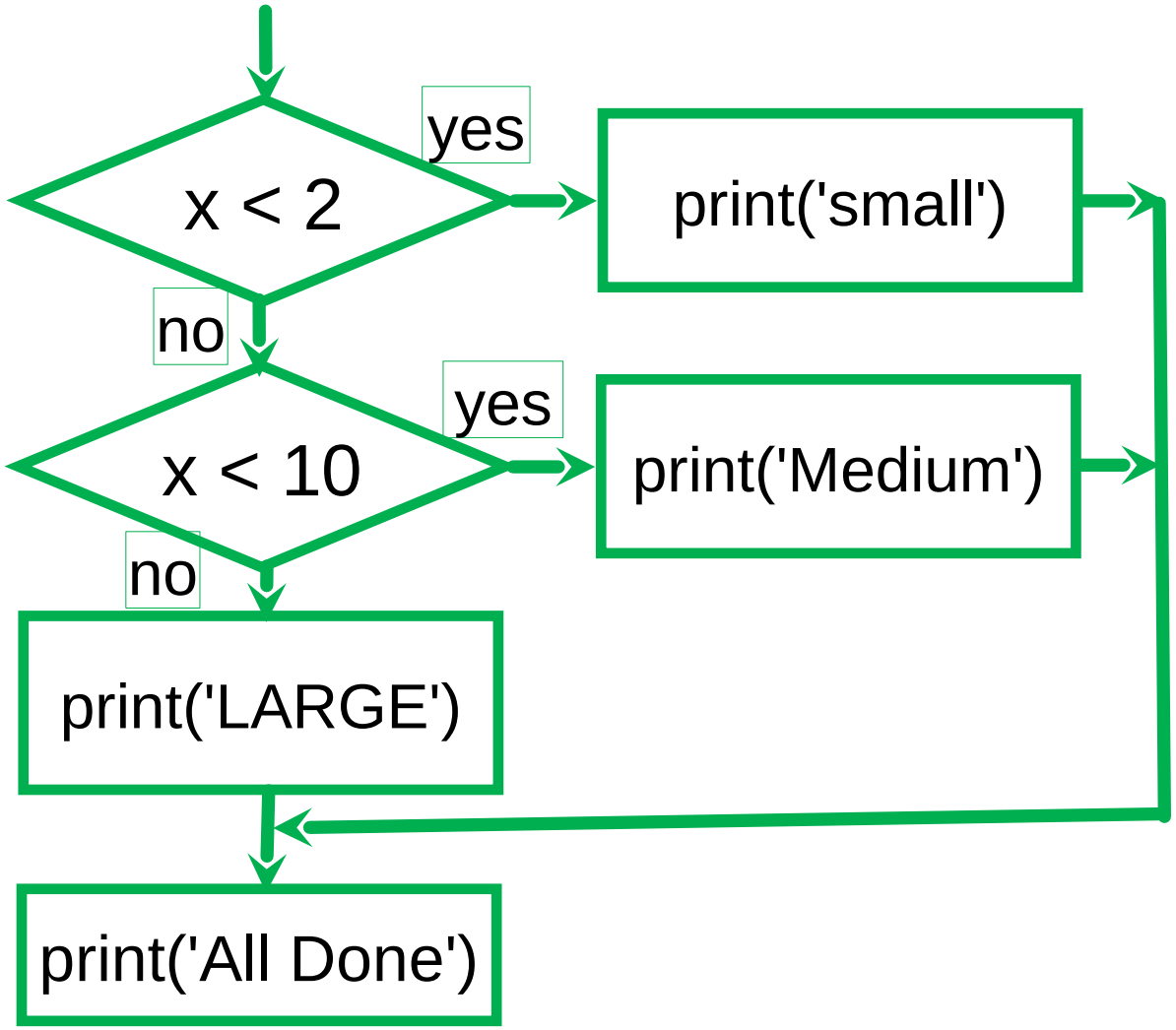
Nested Decisions

```
x = 42
if x > 1 :
    print('More than one')
    if x < 100 :
        print('Less than 100')
print('All done')
```



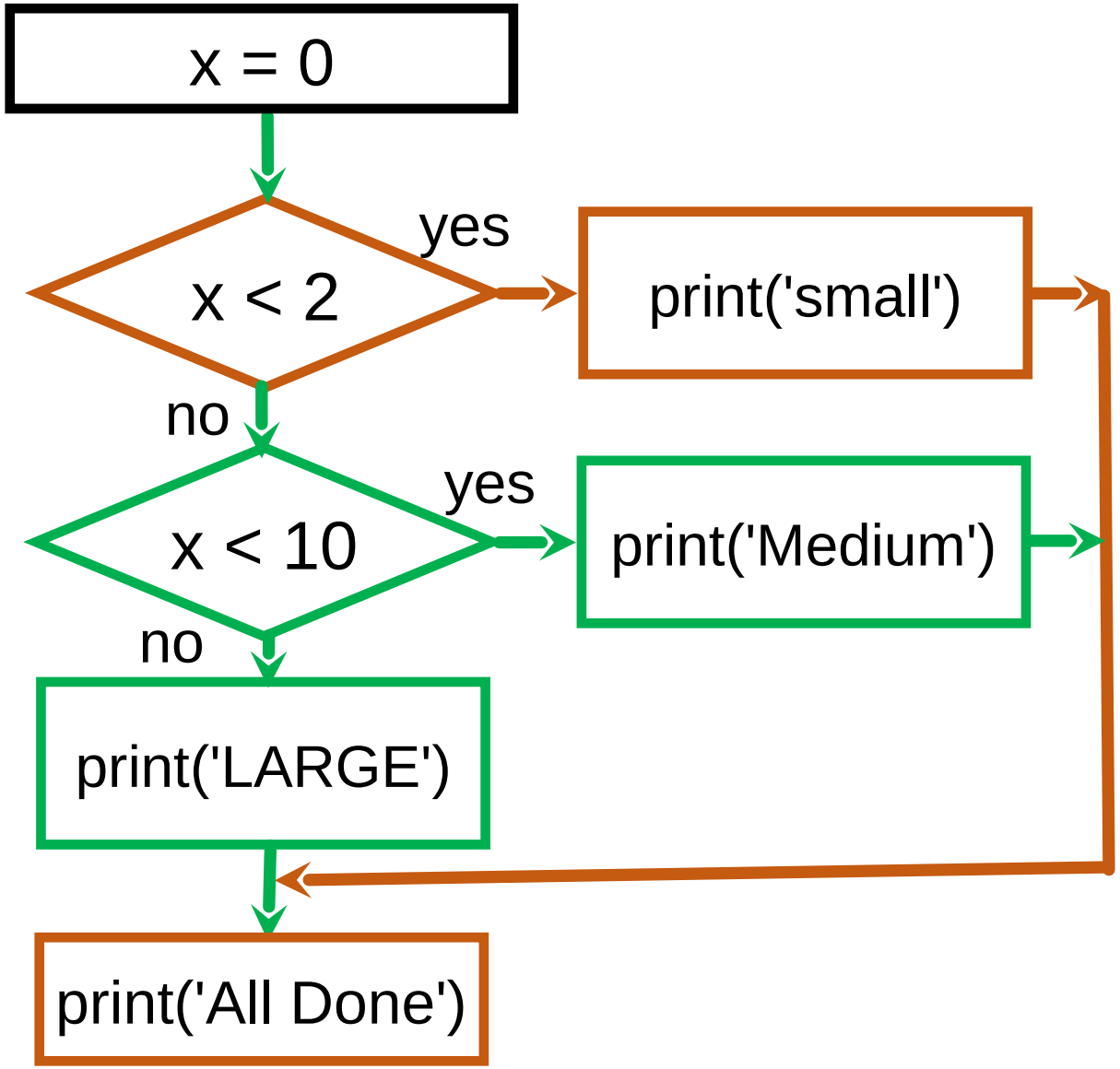
Multi-way

```
if x < 2 :
    print('small')
elif x < 10 :
    print('Medium')
else :
    print('LARGE')
print('All done')
```



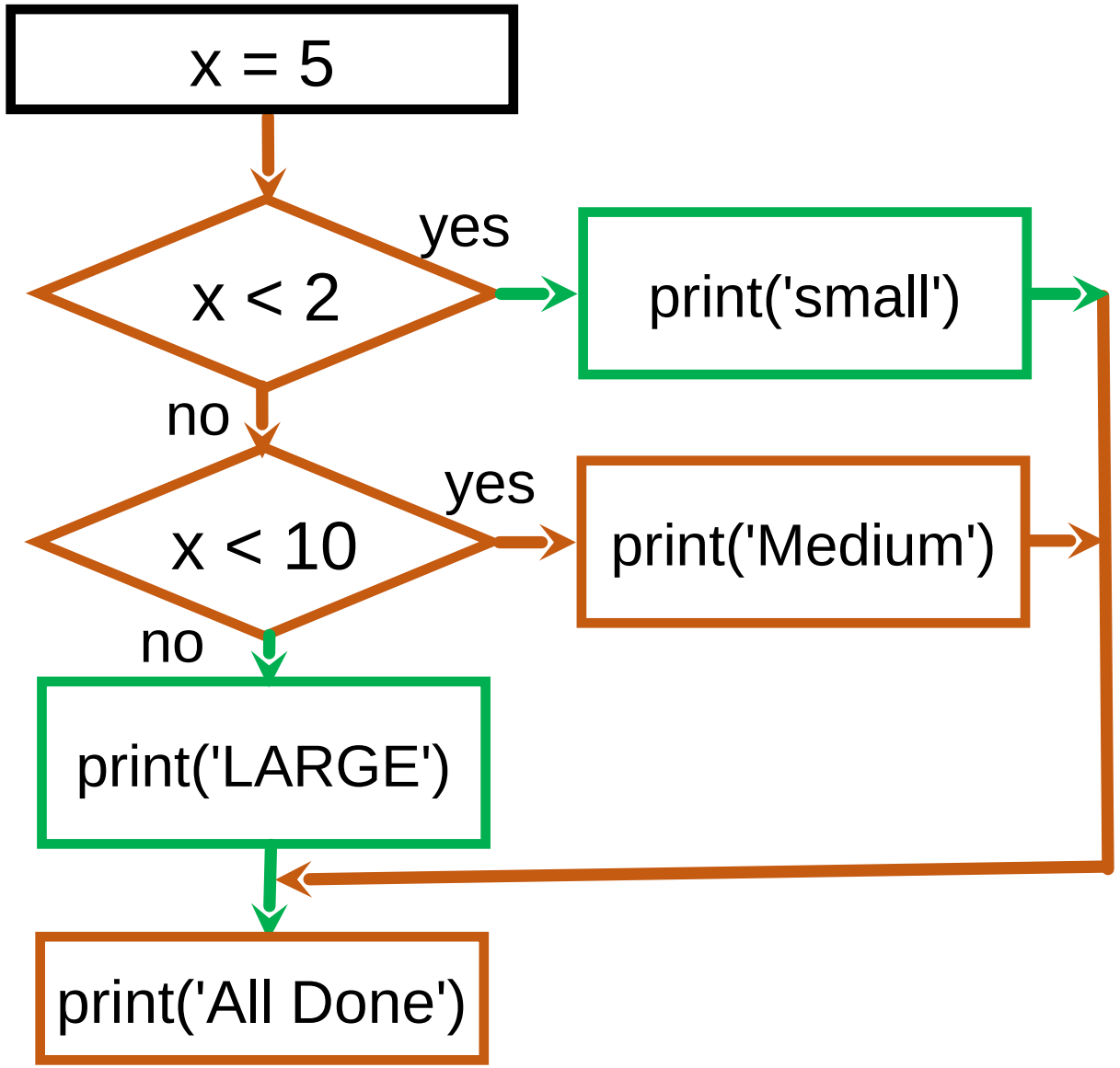
Multi-way

```
x = 0
if x < 2 :
    print('small')
elif x < 10 :
    print('Medium')
else :
    print('LARGE')
print('All done')
```



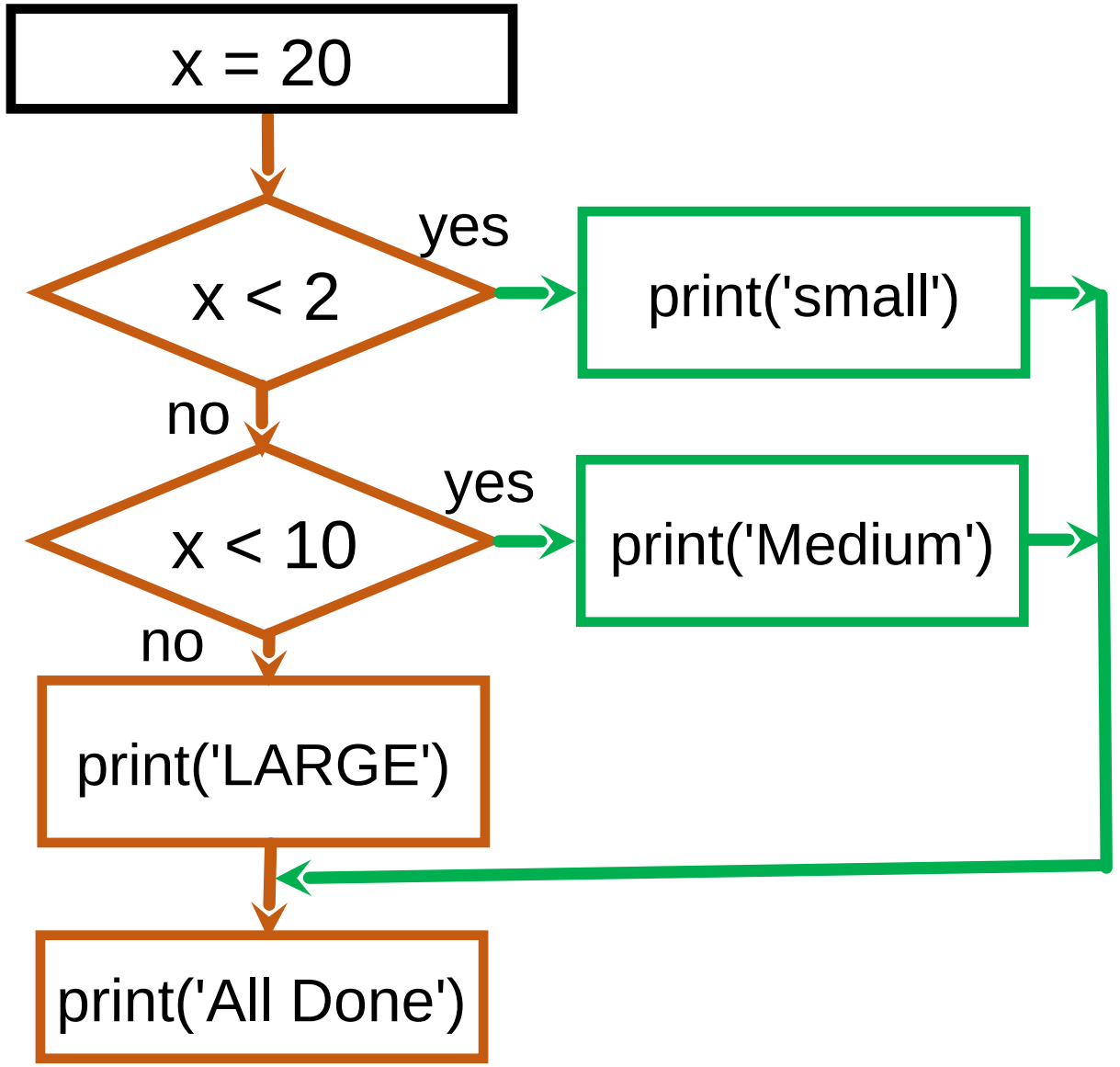
Multi-way

```
x = 5
if x < 2 :
    print('small')
elif x < 10 :
    print('Medium')
else :
    print('LARGE')
print('All done')
```



Multi-way

```
x = 20
if x < 2 :
    print('small')
elif x < 10 :
    print('Medium')
else :
    print('LARGE')
print('All done')
```



Multi-way

```
# No Else
x = 5
if x < 2 :
    print('Small')
elif x < 10 :
    print('Medium')

print('All done')
```

```
if x < 2 :
    print('Small')
elif x < 10 :
    print('Medium')
elif x < 20 :
    print('Big')
elif x < 40 :
    print('Large')
elif x < 100:
    print('Huge')
else :
    print('Ginormous')
```

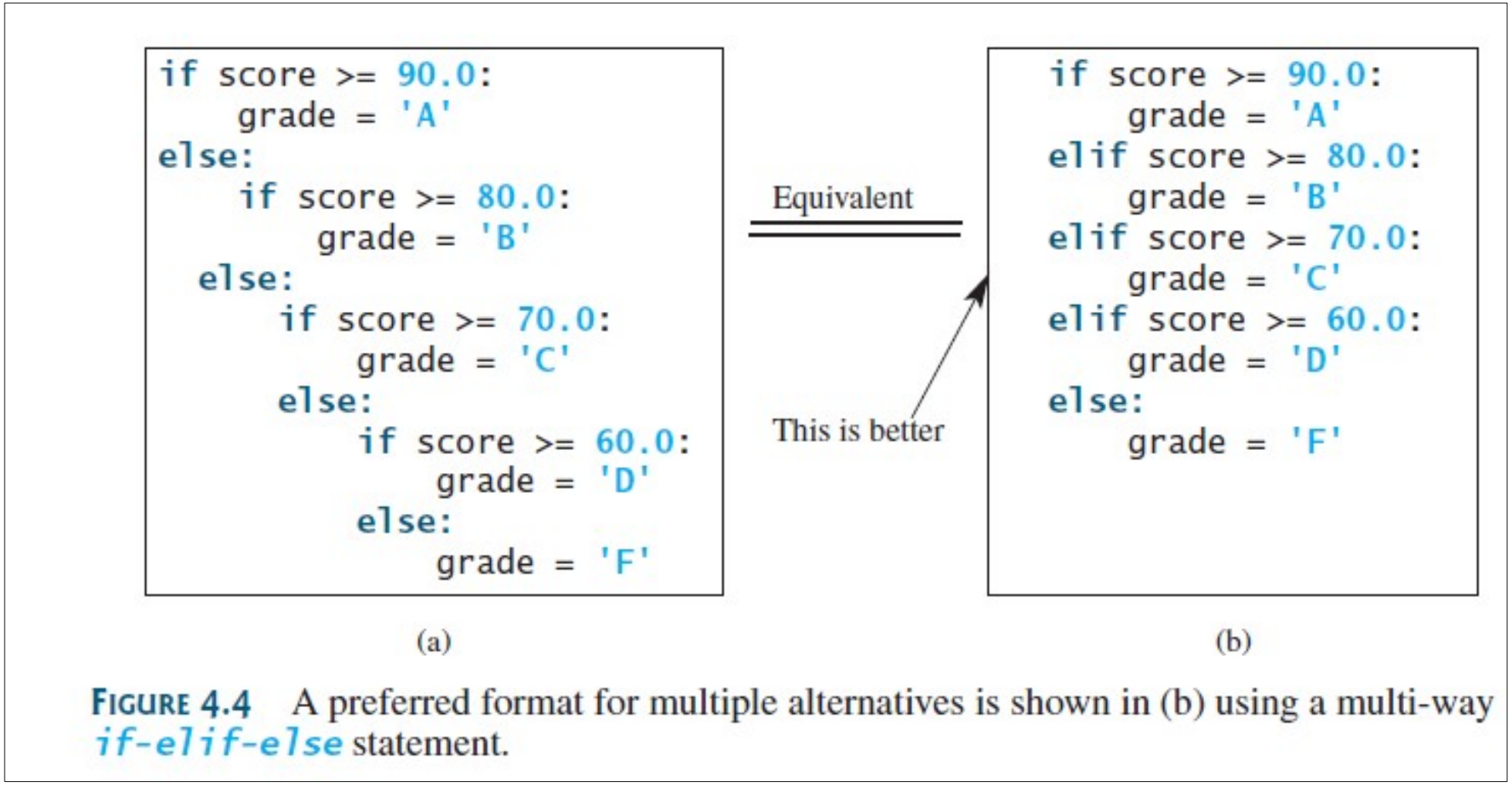
Multi-way Puzzles

Which will never print regardless of the value for x?

```
if x < 2 :
    print('Below 2')
elif x >= 2 :
    print('Two or more')
else :
    print('Something else')
```

```
if x < 2 :
    print('Below 2')
elif x < 20 :
    print('Below 20')
elif x < 10 :
    print('Below 10')
else :
    print('Something else')
```

Multiple Alternative if Statements



```
if score >= 90.0:
    grade = 'A'
else:
    if score >= 80.0:
        grade = 'B'
    else:
        if score >= 70.0:
            grade = 'C'
        else:
            if score >= 60.0:
                grade = 'D'
            else:
                grade = 'F'
```

(a)

Equivalent

This is better

```
if score >= 90.0:
    grade = 'A'
elif score >= 80.0:
    grade = 'B'
elif score >= 70.0:
    grade = 'C'
elif score >= 60.0:
    grade = 'D'
else:
    grade = 'F'
```

(b)

FIGURE 4.4 A preferred format for multiple alternatives is shown in (b) using a multi-way *if-elif-else* statement.

Figure 4.4 Explanation

- The **if** statement in **Figure 4.4a** is **equivalent** to the if statement in **Figure 4.4b**.
- Figure 4.4b is the **preferred coding style** for multiple alternative if statements.
  - This style, called **multiway if statements**
  - **Avoids deep indentation**
  - and makes the program easier to read.
- The **multi-way if** statements uses the syntax **if-elif-else**;
- elif (short for **else if**) is a **Python keyword**.

Multiple Alternative if Statements

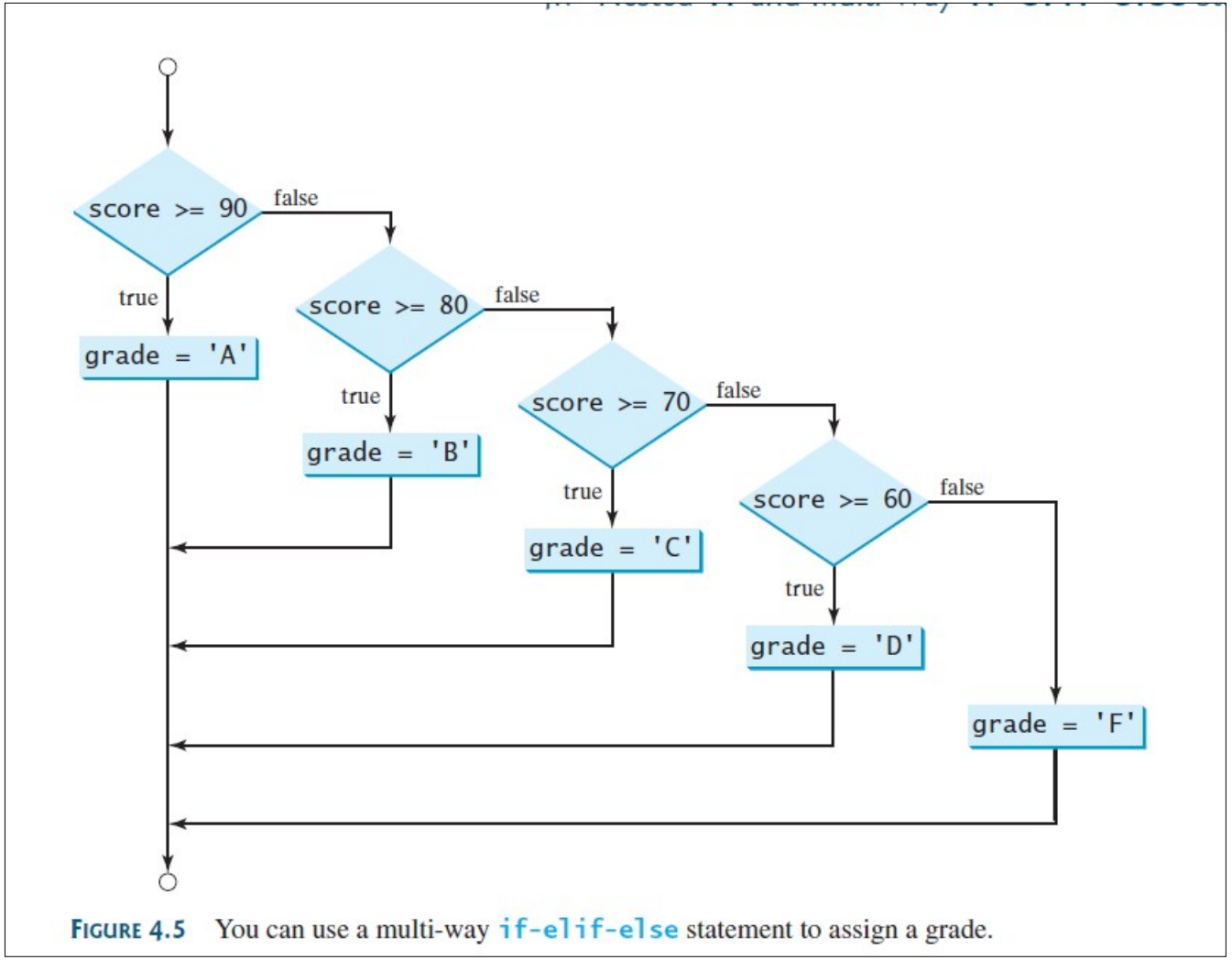


FIGURE 4.5 You can use a multi-way `if-elif-else` statement to assign a grade.

animation

Trace if-else statement

Suppose score is 70.0

The condition is false

```
if score >= 90.0:
    grade = 'A'
elif score >= 80.0:
    grade = 'B'
elif score >= 70.0:
    grade = 'C'
elif score >= 60.0:
    grade = 'D'
else:
    grade = 'F'
```

animation

Trace if-else statement

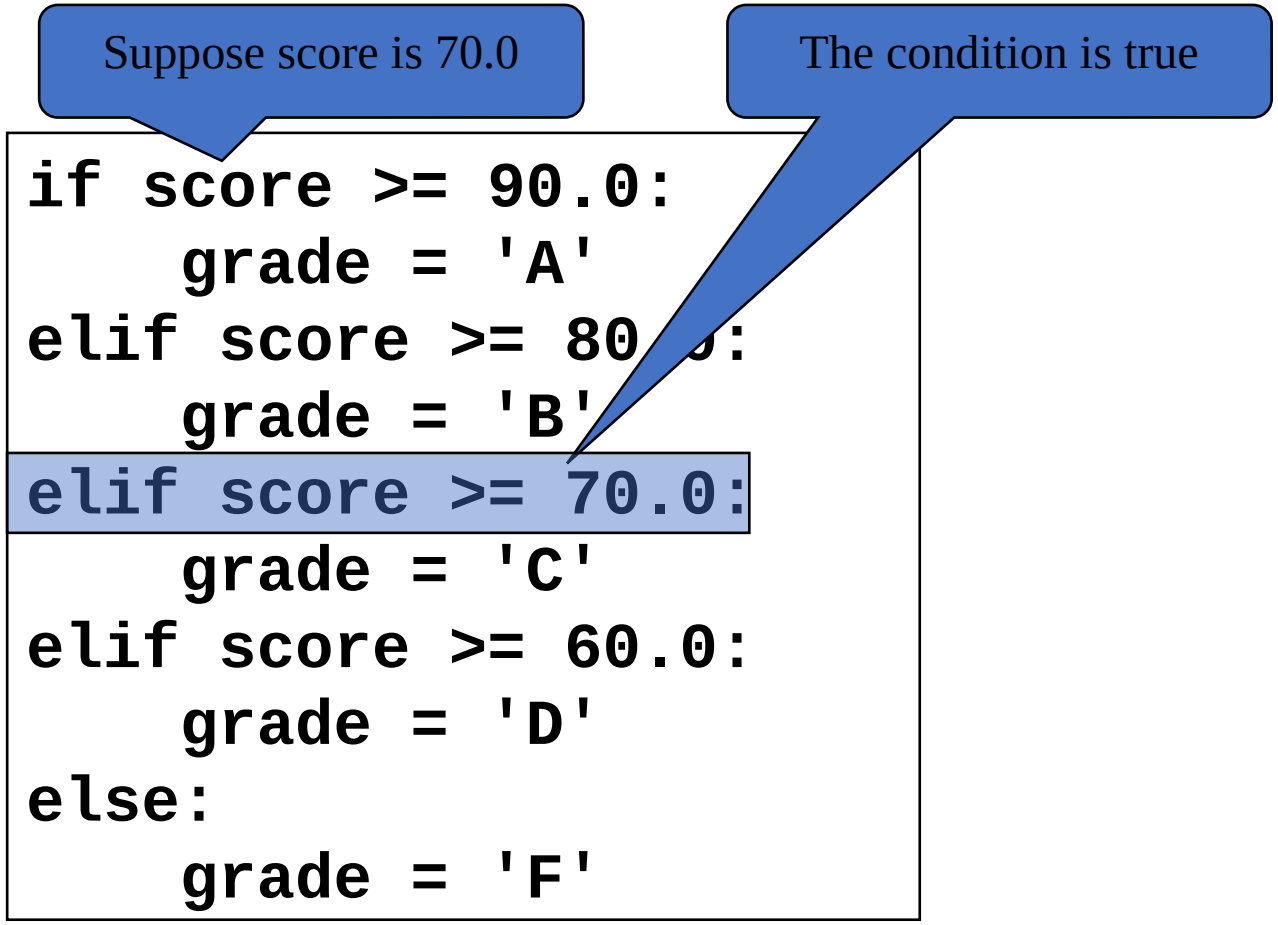
Suppose score is 70.0

The condition is false

```
if score >= 90.0:
    grade = 'A'
elif score >= 80.0:
    grade = 'B'
elif score >= 70.0:
    grade = 'C'
elif score >= 60.0:
    grade = 'D'
else:
    grade = 'F'
```

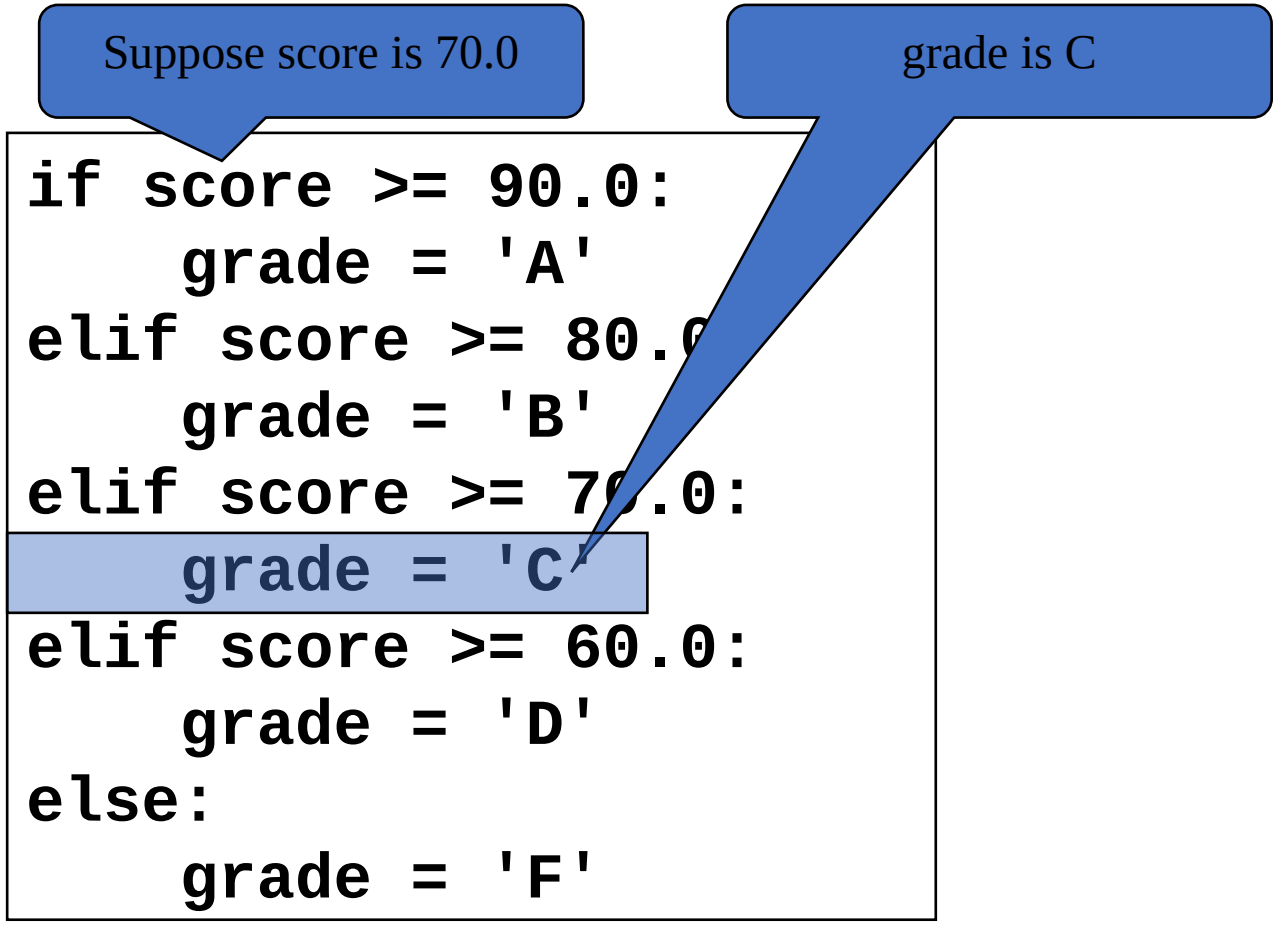
animation

Trace if-else statement



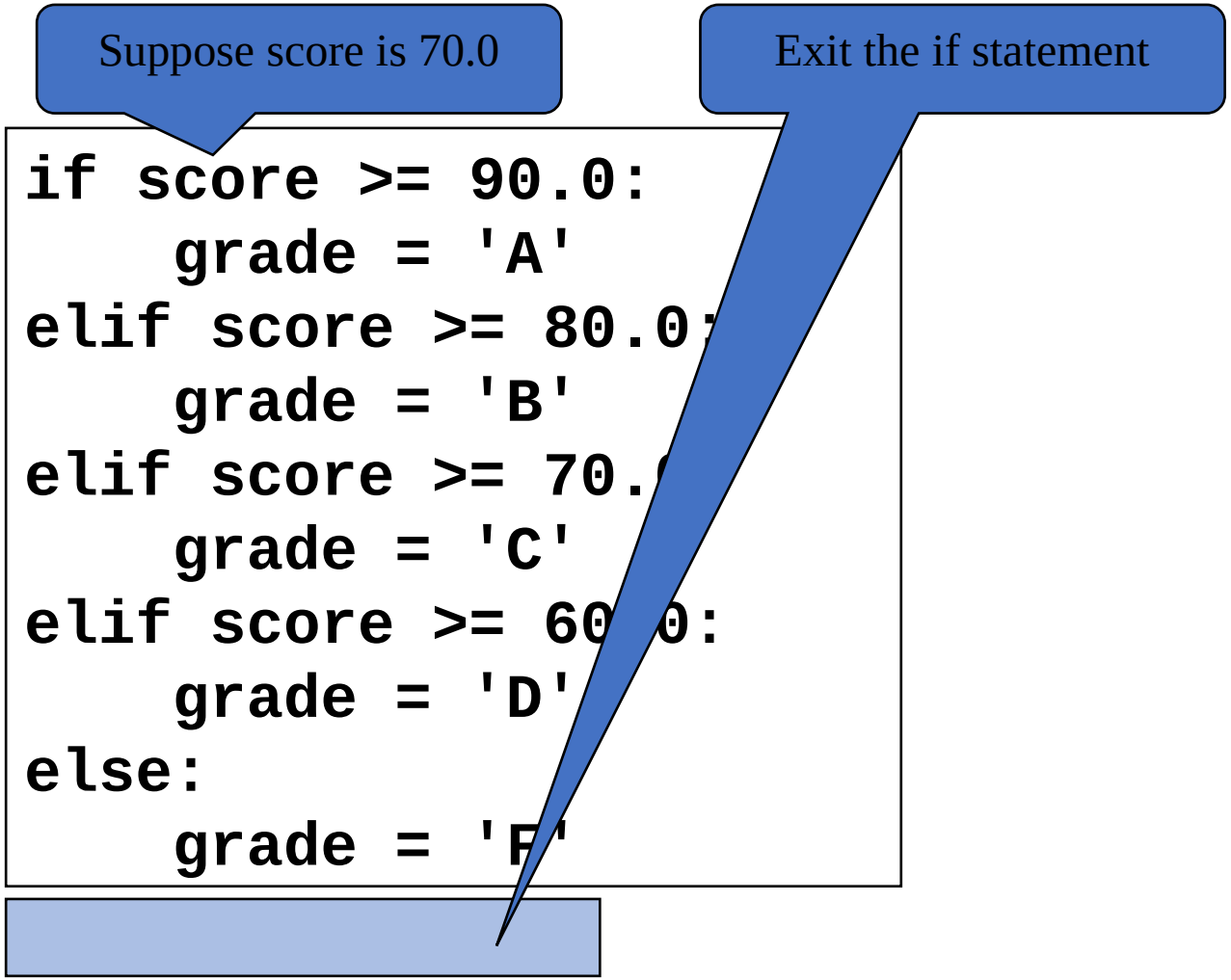
animation

Trace if-else statement



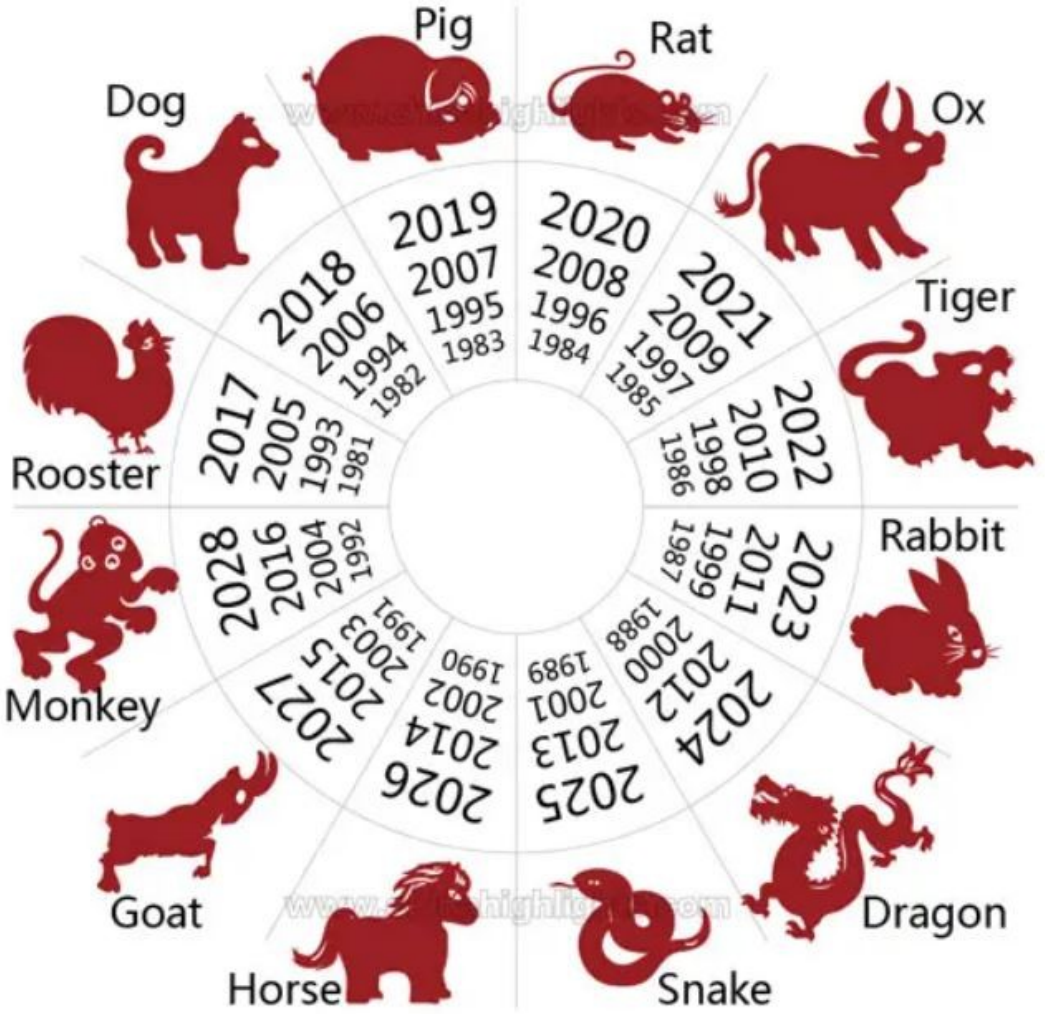
animation

Trace if-else statement



Example

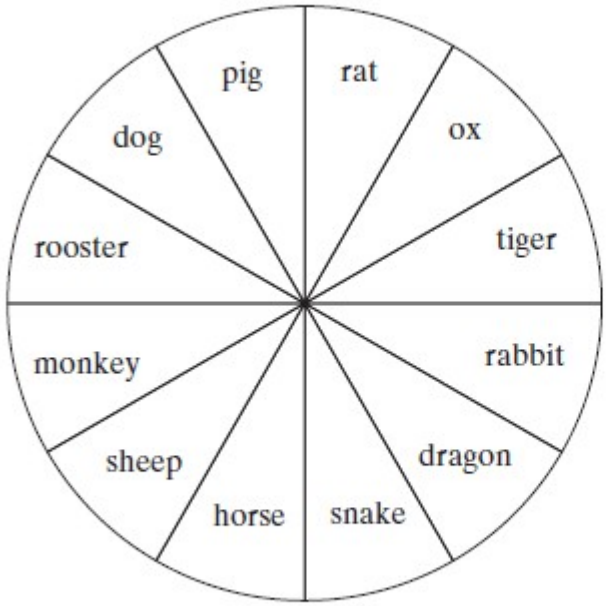
- Now let us write a program to find out the Chinese Zodiac sign for a given year. The Chinese Zodiac sign is based on a **12-year cycle**, each year being represented by an animal: rat, ox, tiger, rabbit, dragon, snake, horse, sheep, monkey, rooster, dog, and pig, in this cycle.



Example

```
Enter a year: 1963 ↵Enter
rabbit
```

```
Enter a year: 1877 ↵Enter
ox
```



year % 12 =
0: monkey
1: rooster
2: dog
3: pig
4: rat
5: ox
6: tiger
7: rabbit
8: dragon
9: snake
10: horse
11: sheep

FIGURE 4.6 The Chinese zodiac is based on a 12-year cycle.

4.8 Common Errors in Selection Statements

Most common errors in selection statements are caused by incorrect indentation.

Common Errors

- Most common errors in selection statements are caused by **incorrect indentation**. Consider the following code in (a) and (b).

```
radius = -20

if radius >= 0:
    area = radius * radius * math.pi
print("The area is", area)
```

(a) Wrong

```
radius = -20

if radius >= 0:
    area = radius * radius * math.pi
    print("The area is", area)
```

(b) Correct

In (a), the print statement is not in the if block. To place it in the if block, you have to indent it, as shown in (b).

Another Example

- Consider another example in the following code in (a) and (b). The code in (a) below has two if clauses and one else clause. Which if clause is matched by the else clause? The indentation indicates that the else clause matches the first if clause in (a) and the second if clause in (b).

```
i = 1
j = 2
k = 3

if i > j:
    if i > k:
        print('A')
else:
    print('B')
```

(a)

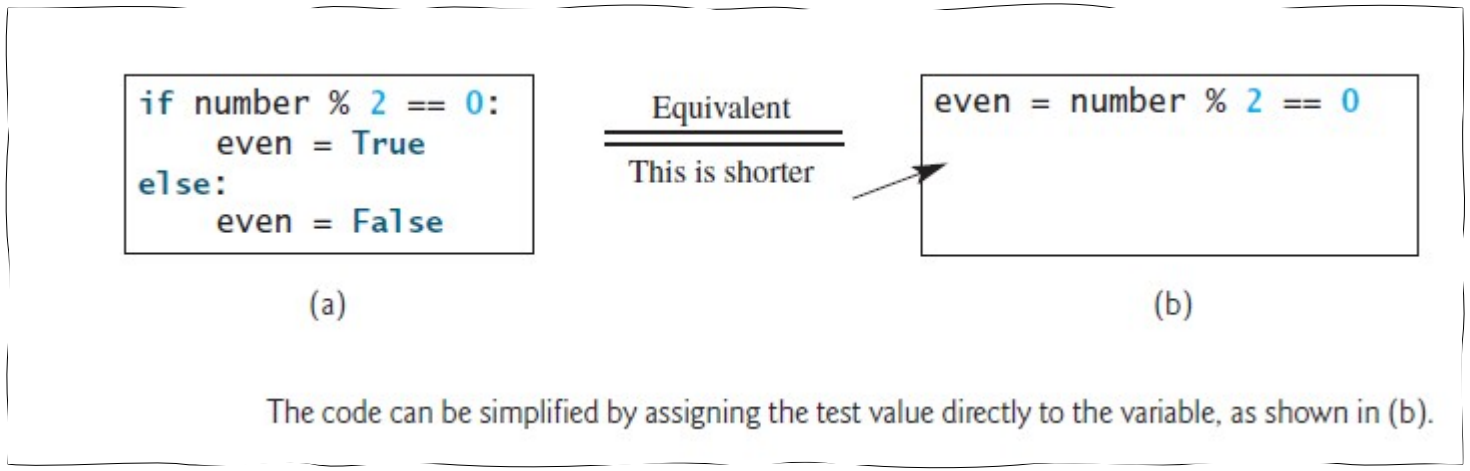
```
i = 1
j = 2
k = 3

if i > j:
    if i > k:
        print('A')
    else:
        print('B')
```

(b)

TIP

- Often new programmers write the code that assigns a test condition to a Boolean variable, like the code in (a):



4.9 Case Study: Computing Body Mass Index

You can use nested if *statements to write a program that interprets body mass index*

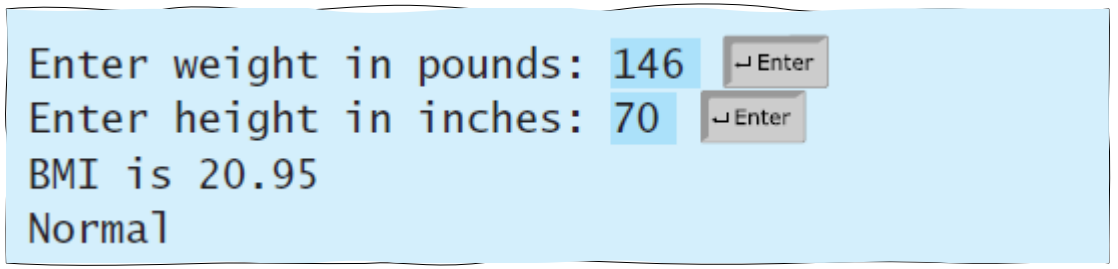
Body Mass Index (BMI)

- Body Mass Index (BMI) is a measure of health on weight. It can be calculated by taking your weight in kilograms and dividing by the square of your height in meters. The interpretation of BMI for people 16 years or older is as follows:

BMI	Interpretation
Below 18.5	Underweight
18.5–24.9	Normal
25.0–29.9	Overweight
Above 30.0	Obese

Case Study: Computing Body Mass Index

- Write a program that prompts the user to enter a weight in pounds and height in inches and then displays the BMI.
- Note that
  - one pound is 0.45359237 kilograms
  - one inch is 0.0254 meters.



ComputeBMI.py

- **Step 1**: Prompt the user to enter weight in pounds

```
# Prompt the user to enter weight in pounds
weight = eval(input("Enter weight in pounds: "))
```

- **Step 2**: Prompt the user to enter height in inches

```
# Prompt the user to enter height in inches
height = eval(input("Enter height in inches: "))
```

ComputeBMI.py

- **Step 3**: Compute BMI

```
KILOGRAMS_PER_POUND = 0.45359237  # Constant
METERS_PER_INCH = 0.0254          # Constant

# Compute BMI
weightInKilograms = weight * KILOGRAMS_PER_POUND
heightInMeters = height * METERS_PER_INCH
bmi = weightInKilograms / (heightInMeters * heightInMeters)
```

- **Step 4**: print the results

```
# Display result
print("BMI is", format(bmi, ".2f"))
if bmi < 18.5:
    print("Underweight")
elif bmi < 25:
    print("Normal")
elif bmi < 30:
    print("Overweight")
else:
    print("Obese")
```

4.10 Case Study: Computing Taxes

*You can use **nested** **if** statements to write a program for computing taxes.*

Case Study: Computing Taxes

- The US federal personal income tax is calculated based on the filing status and taxable income. **There are four filing statuses**: single filers, married filing jointly, married filing separately, and head of household. The tax rates for 2009 are shown below

TABLE 4.2 2009 U.S. Federal Personal Tax Rates

Marginal Tax Rate	*Single*	*Married Filing Jointly*	*Married Filing Separately*	*Head of Household*
10%	$0 – $8,350	$0 – $16,700	$0 – $8,350	$0 – $11,950
15%	$8,351 – $33,950	$16,701 – $67,900	$8,351 – $33,950	$11,951 – $45,500
25%	$33,951 – $82,250	$67,901 – $137,050	$33,951 – $68,525	$45,501 – $117,450
28%	$82,251 – $171,550	$137,051 – $208,850	$68,526 – $104,425	$117,451 – $190,200
33%	$171,551 – $372,950	$208,851 – $372,950	$104,426 – $186,475	$190,201 – $372,950
35%	$372,951+	$372,951+	$186,476+	$372,951+

Case Study: Computing Taxes

- Write a program to compute personal income tax. Your program should prompt the user to enter the filing status and taxable income and then compute the tax. Enter 0 for single filers, 1 for married filing jointly, 2 for married filing separately, and 3 for head of household.

```
(0-single filer, 1-married jointly,
2-married separately, 3-head of household)
Enter the filing status: 0 Enter
Enter the taxable income: 400000 Enter
Tax is 117683.50
```

ComputeTax.py

- **Step 1**: Prompt the user to enter filing status

```
# Prompt the user to enter filing status
status = eval(input(
    "(0-single filer, 1-married jointly,\n" +
    "2-married separately, 3-head of household)\n" +
    "Enter the filing status: "))
```

- **Step 2**: Prompt the user to enter taxable income

```
# Prompt the user to enter taxable income
income = eval(input("Enter the taxable income: "))
```

ComputeTax.py

- **Step 3**: Compute Tax
- To do this we need to use nested if → the outer if statement will be:

```
if status == 0:
    # Compute tax for single filers
elif status == 1:
    # Compute tax for married filing jointly
elif status == 2:
    # Compute tax for married filing separately
elif status == 3:
    # Compute tax for head of household
else:
    # Display wrong status
```

Sys Module

- sys.exit() is defined in the sys module.
  - Invoking this function terminates the program.

Tip

- For all programs, you should **write a small amount of code and test it** before moving on to add more code.
- This is called **incremental development and testing**.
- This approach makes **debugging easier**, *because* the errors are likely in the new code you just added.

4.11 Logical Operators

The logical operators not*,* and*, and* or *can be used to create a composite condition.*

Logical Operators & Compound Conditions in Python

- Sometimes a single condition is not enough.
- **Example:**
  - “Apply tax if income is between 10,000 and 20,000”
- This requires **more than one condition**
- A **compound condition** combines multiple conditions into one expression at the same time.
- To combine these conditions we need a logical operators.

Logical Operators

- **Logical operators**, also known as **Boolean operators**, operate on Boolean values to create a new Boolean value.

TABLE 4.3 Boolean Operators

Operator	*Description*
`not`	logical negation
`and`	logical conjunction
`or`	logical disjunction

Not Operator

- A logical negation
  - Return the opposite Boolean of the condition.

TABLE 4.4 Truth Table for Operator `not`

p	not p	*Example (assume* `age = 24, gender = 'F'`*)*
`True`	`False`	`not (age > 18)` is `False`, because `(age > 18)` is `True`.
`False`	`True`	`not (gender == 'M')` is `True`, because `(gender == 'M')` is `False`.

And Operator

- Logical conjunction
- Returns true if both operands are true.
- If one of the operands of an and operator is False, the expression is False.

TABLE 4.5 Truth Table for Operator `and`

p_1	p_2	p_1 and p_2	*Example (assume* `age = 24, gender = 'F'`*)*
`False`	`False`	`False`	`(age > 18) and (gender == 'F')` is `True`, because `(age > 18)` and `(gender == 'F')` are both `True`.
`False`	`True`	`False`	
`True`	`False`	`False`	`(age > 18) and (gender != 'F')` is `False`, because `(gender != 'F')` is `False`.
`True`	`True`	`True`	

OR Operator

- logical disjunction:
  - **True**: if one of the two operands is true

TABLE 4.6 Truth Table for Operator `or`

p_1	p_2	p_1 and p_2	*Example (assume* `age = 24, gender = 'F'`*)*
`False`	`False`	`False`	`(age > 34) or (gender == 'F')` is `True`, because `(gender == 'F')` is `True`.
`False`	`True`	`True`	
`True`	`False`	`True`	`(age > 34) or (gender == 'M')` is `False`, because `(age > 34)` and `(gender == 'M')` are both `False`.
`True`	`True`	`True`	

TestBooleanOperators.py

- Write a program that checks:
  - whether a number is divisible by 2 **and** 3,
  - whether a number is divisible by 2 **or** 3,
  - and whether a number is divisible by 2 **or** 3 **but not both**

```
Enter an integer: 18 ↵Enter
18 is divisible by 2 and 3
18 is divisible by 2 or 3
```

```
Enter an integer: 15 ↵Enter
15 is divisible by 2 or 3
15 is divisible by 2 or 3, but not both
```

TestBooleanOperators.py

- divisible by 2 **and** 3

```
if number % 2 == 0 and number % 3 == 0:
    print(number, "is divisible by 2 and 3")
```

- divisible by 2 **or** 3

```
if number % 2 == 0 or number % 3 == 0:
    print(number, "is divisible by 2 or 3")
```

- divisible by 2 **or** 3 but not both

```
if (number % 2 == 0 or number % 3 == 0) and  \
       not (number % 2 == 0 and number % 3 == 0):
    print(number, "is divisible by 2 or 3, but not both")
```

Short-Circuit Evaluation

- Python does NOT always evaluate both operands.
- It stops early when result is already known.
- Syntax: p1 **and** p2
- Behavior:
  - If p1 is False → result is False immediately.
  - If p1 is True → evaluate p2.
- Syntax: p1 **or** p2
- Behavior:
  - If p1 is True → result is True immediately
  - If p1 is False → evaluate p2

Short-Circuit Evaluation

- **Why This Matters**
  - Improves **performance**
  - Avoids unnecessary computations
  - Prevents errors in some cases

```
>>> x=0
...
>>> if x>0 and 10/x >1 :
...     print("Good")
...
```

```
>>> x= 0
...
>>> if 10/x >1 and  x>0 :
...     print("fail")
...
...
Traceback (most recent call last):
  File "<pyshell#16>", line 1, in <module>
    if 10/x >1 and  x>0 :
ZeroDivisionError: division by zero
>>>
```

4.12 Case Study: Determining Leap Years

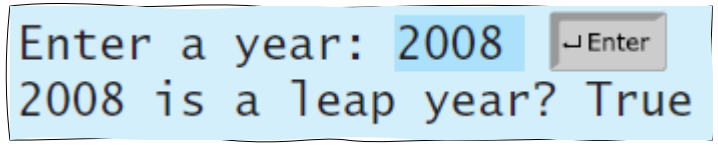
Problem: Determining Leap Year?

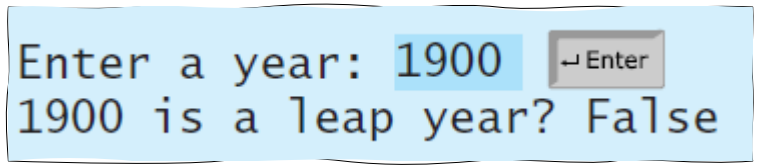
- This program first prompts the user to enter a year as an **int** value and checks if it is a leap year.
- A year is a **leap year** if it is divisible by 4 but not by 100, or it is divisible by 400.
- (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0)

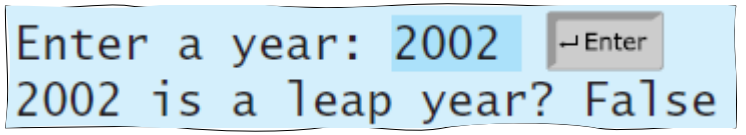
LeapYear.py

```
year = eval(input("Enter a year: "))

# Check if the year is a leap year
isLeapYear =(year % 4 == 0 and year % 100 != 0) or \
      (year % 400 == 0)
# Display the result
print(year, "is a leap year?", isLeapYear)
```







4.13 Case Study: Lottery

Problem: Lottery

- Write a program that **randomly** generates a lottery of a **two-digit** number, **prompts** the user to **enter a two-digit** number, and determines whether the user wins according to the following rule:
  - If the user input matches the lottery in exact order, the award is $10,000.
  - If the user input matches the lottery, the award is $3,000.
  - If one digit in the user input matches a digit in the lottery, the award is $1,000.

```
Enter your lottery pick (two digits): 45 ↵Enter
The lottery number is 12
Sorry, no match
```

```
Enter your lottery pick (two digits): 23 ↵Enter
The lottery number is 34
Match one digit: you win $1,000
```

Lottery.py

- **Step 1**: Generate a lottery number

```
import random

# Generate a lottery number
lottery = random.randint(0, 99)
```

- **Step 2**: Prompt the user to enter a guess

```
# Prompt the user to enter a guess
guess = eval(input("Enter your lottery pick (two digits): "))
```

- **Step 3**: Get digits from lottery

```
# Get digits from lottery
lotteryDigit1 = lottery // 10
lotteryDigit2 = lottery % 10
```

Lottery.py

- **Step 4**: Get digits from guess

```
# Get digits from guess
guessDigit1 = guess // 10
guessDigit2 = guess % 10
```

- **Step 5**: Check the guess

```
print("The lottery number is", lottery)

# Check the guess
if guess == lottery:
    print("Exact match: you win $10,000")
elif (guessDigit2 == lotteryDigit1 and \
    guessDigit1 == lotteryDigit2):
    print("Match all digits: you win $3,000")
elif (guessDigit1 == lotteryDigit1
        or guessDigit1 == lotteryDigit2
        or guessDigit2 == lotteryDigit1
        or guessDigit2 == lotteryDigit2):
    print("Match one digit: you win $1,000")
else:
    print("Sorry, no match")
```

4.14 Conditional Expressions

Conditional Expressions

- A conditional expression evaluates an expression based on a condition.

```
if x > 0:
    y = 1
else:
    y = -1
```

is equivalent to

```
y = 1 if x > 0 else -1
```

expression1 **if** boolean-expression **else** expression2

Conditional Operator

```
if num % 2 == 0:
   print(str(num) + “is even”)
else:
   print(str(num) + “is odd”);
```

```
print("number is even" if number % 2 == 0 else "number is odd")
```

4.15 Operator Precedence and Associativity

Operator precedence and associativity determine the order in which operators are evaluated.

Operator precedence

- **Operator precedence** and **operator associativity** determine the order in which Python evaluates operators.

TABLE 4.7 Operator Precedence Chart

Precedence	*Operator*
↓	+, - (Unary plus and minus)
	** (Exponentiation)
	not
	*, /, //, % (Multiplication, division, integer division, and remainder)
	+, - (Binary addition and subtraction)
	<, <=, >, >= (Comparison)
	==, != (Equality)
	and
	or
	=, +=, -=, *=, /=, //=, %= (Assignment operators)

Operator Precedence and Associativity

The expression in the parentheses is evaluated first. (Parentheses can be nested, in which case the expression in the inner parentheses is executed first.) When evaluating an expression without parentheses, the operators are applied according to the precedence rule and the associativity rule.

If operators with the same precedence are next to each other, their associativity determines the order of evaluation. All binary operators except assignment operators are left-associative.

Operator Associativity

When two operators with the same precedence are evaluated, the *associativity* of the operators determines the order of evaluation.

All binary operators except assignment operators are *left-associative*.

a – b + c – d is equivalent to ((a – b) + c) – d

Assignment operators are *right-associative*. Therefore, the expression

a = b += c = 5 is equivalent to a = (b += (c = 5))

End of Chapter 04